

Client Primitives Agenda

- Elicitation — form mode and URL mode

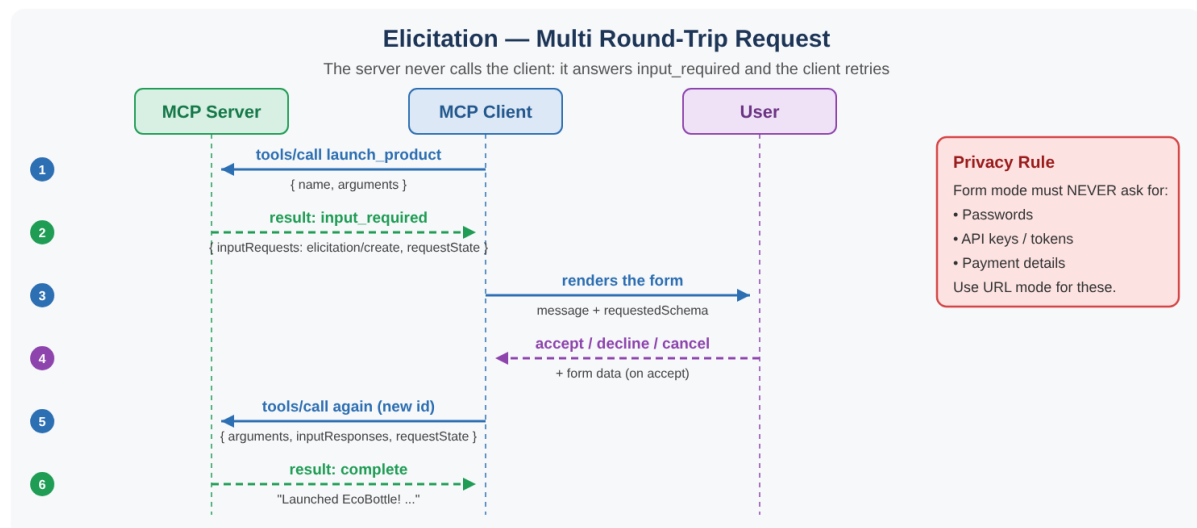
Elicitation

Elicitation lets a server ask the user for specific information mid-call, instead of requiring everything upfront or failing when data is missing.

It has two modes:

- Form mode** — the server sends a message and a small, flat JSON Schema; the client renders a form; the user accepts, declines or cancels.
- URL mode** — the server sends a URL; the client shows it, asks for consent and opens it in the browser. Use it for sign-ins, payments and anything sensitive: the data never passes through the client.

The Flow



Example — Product Launch Confirmation

Inside the `input_required` result, the server's elicitation request looks like this (captured from `product_server.py` below; Pydantic "title" keys omitted):

```
{
  "method": "elicitation/create",
  "params": {
    "mode": "form",
    "message": "Please confirm launch details for 'EcoBottle':",
    "requestedSchema": {
      "type": "object",
      "properties": {
        "confirmLaunch": {
          "type": "boolean",
```

```
"description": "Confirm you want to launch this product"
},
"price": {
  "type": "number", "default": 29.99, "minimum": 0,
  "description": "Retail price (USD)"
},
"initialStock": {
  "type": "integer", "default": 100, "minimum": 0,
  "description": "Initial stock quantity"
},
"shippingSpeed": {
  "type": "string", "default": "standard",
  "enum": ["standard", "priority", "overnight"],
  "description": "Shipping speed to offer at launch"
}
},
"required": ["confirmLaunch"]
}
}
}
```

The Response

The client answers with one of three actions:

action value	Meaning
accept	User filled in the form and confirmed — content holds the answers
cancel	User dismissed the dialog without choosing (closed it, pressed Esc)
decline	User explicitly said no

Server code — product_server.py

```
"""
product_server.py - an MCP server that asks the user for input mid-call (elicitation).
"""

import sys
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from mcp.server import MCPServer
from mcp.server.elicitation import ElicitationResult
from mcp.server.mcpserver import Elicit, Resolve
```

```

mcp = MCPServer("product server")

def log(msg: str) -> None:
    """Narrate to STDERR (stdout is reserved for the MCP protocol)."""
    print(f"[SERVER] {msg}", file=sys.stderr, flush=True)

# The form the user will see. MCPServer turns this Pydantic model into the
# requestedSchema of the elicitation/create request (flat, primitive fields only).

class LaunchConfirmation(BaseModel):
    confirmLaunch: bool = Field(description="Confirm you want to launch this product")
    price: float = Field(default=29.99, ge=0, description="Retail price (USD)")
    initialStock: int = Field(default=100, ge=0, description="Initial stock quantity")
    shippingSpeed: Literal["standard", "priority", "overnight"] = Field(default="standard",
description="Shipping speed to offer at launch")

# A resolver: says WHAT to ask. It can read tool arguments by name.
def ask_launch_details(product_name: str) -> Elicit[LaunchConfirmation]:
    log(f"asking the user to confirm launch details for '{product_name}'")
    return Elicit(f"Please confirm launch details for '{product_name}':", LaunchConfirmation)

@mcp.tool()
async def launch_product(
    product_name: str,
    answer: Annotated[ElicitationResult[LaunchConfirmation], Resolve(ask_launch_details)],
) -> str:
    """Launch a product, confirming pricing and inventory with the user first."""
    # By the time the body runs, the SDK has already asked the user
    # (input_required -> client retry) and injected the outcome here.
    if answer.action == "accept":
        data = answer.data
        if not data.confirmLaunch:
            return "Launch not confirmed by user."
        log("user accepted, finalizing launch")
        return (
            f"Launched {product_name}! Price: ${data.price:.2f}, "
            f"Initial stock: {data.initialStock}, Shipping: {data.shippingSpeed}."
        )

```

```

if answer.action == "decline":
    return "User declined to provide launch details."
return "User cancelled the launch flow."

if __name__ == "__main__":
    log("product-server starting on stdio...")
    mcp.run() # stdio transport by default

```

How the server code works:

- **Elicit(message, Model)** — returned by a resolver function; it describes the question. The Pydantic model becomes the requestedSchema (flat fields only: string, number, integer, boolean, enum).
- **Annotated[..., Resolve(fn)]** — tells MCPServer to run the resolver before the tool body. On a modern connection the SDK answers input_required, waits for the client's retry, and injects the outcome into answer.
- **ElicitationResult[Model]** — gives you the full outcome so you can branch on accept, decline and cancel. Annotate the parameter with the plain model instead and a decline or cancel simply stops the call.

Client code — product_client.py

```

import asyncio
import sys
from mcp import Client, StdioServerParameters
from mcp.client import ClientRequestContext
from mcp.types import ElicitRequestParams, ElicitRequestURLParams, ElicitResult, TextContent

async def handle_elicitation(context: ClientRequestContext, params: ElicitRequestParams) -> ElicitResult:
    """Called when a tool result says input_required with an elicitation/create request."""
    print("[CLIENT] the server is asking the user for input.", flush=True)

    if isinstance(params, ElicitRequestURLParams): # URL mode
        print(f" {params.message}\n Open this link to continue: {params.url}")
        ok = input(" Open it? [y/N]: ").strip().lower() == "y"
        return ElicitResult(action="accept" if ok else "decline")

    print(f"\n {params.message}") # form mode
    choice = input(" Proceed? [Enter = fill form, d = decline, c = cancel]: ").strip().lower()
    if choice == "d":

```

```

    return ElicitResult(action="decline")
if choice == "c":
    return ElicitResult(action="cancel")
return ElicitResult(action="accept", content=fill_form(params.requested_schema))

def fill_form(schema: dict) -> dict:
    """Build a console 'form' from the requestedSchema the server sent."""
    answers = {}
    for name, prop in schema["properties"].items():
        label = prop.get("description", name)
        default = prop.get("default")
        if "enum" in prop:
            label += f" {prop['enum']}"
            hint = f" [default: {default}] " if default is not None else ""
            raw = input(f" - {label}{hint}: ").strip()
            if raw == "" and default is not None:
                answers[name] = default
            elif prop["type"] == "boolean":
                answers[name] = raw.lower() in ("y", "yes", "true", "1")
            elif prop["type"] == "integer":
                answers[name] = int(raw)
            elif prop["type"] == "number":
                answers[name] = float(raw)
            else:
                answers[name] = raw
    return answers

async def main() -> None:
    params = StdioServerParameters(command=sys.executable, args=["product_server.py"])
    async with Client(params, elicitation_callback=handle_elicitation) as client:
        print(f"[CLIENT] connected (protocol {client.protocol_version}).")
        name = input("Product name [EcoBottle]: ").strip() or "EcoBottle"
        # The Client runs the input_required -> answer -> retry loop for you.
        result = await client.call_tool("launch_product", {"product_name": name})
        print("\n--- tool result ---")
        print("\n".join(b.text for b in result.content if isinstance(b, TextContent)))

if __name__ == "__main__":
    asyncio.run(main())

```

Run it (the client starts the server for you):

```

pip install "mcp[cli]>=2.0.0"
python product_client.py

# Sample session
Product name [EcoBottle]:
[CLIENT] the server is asking the user for input.
Please confirm launch details for 'EcoBottle':
Proceed? [Enter = fill form, d = decline, c = cancel]:
- Confirm you want to launch this product: y
- Retail price (USD) [default: 29.99]: 49.5
- Initial stock quantity [default: 100]:
- Shipping speed to offer at launch ['standard', 'priority', 'overnight'] [default: standard]: priority
--- tool result ---
Launched EcoBottle! Price: $49.50, Initial stock: 100, Shipping: priority.

```

Making an Elicit call midway in a tool execution:

```

"""
product_server.py - an MCP server that asks the user for input mid-call (elicitation).
"""

import sys
from typing import Literal
from pydantic import BaseModel, Field
from mcp.server import MCPServer
from mcp.server.elicitation import render_elicitation_schema
from mcp.server.mcpserver import Context
from mcp.types import ElicitRequest, ElicitRequestFormParams, InputRequiredResult

mcp = MCPServer("product server")

def log(msg: str) -> None:
    """Narrate to STDERR (stdout is reserved for the MCP protocol)."""
    print(f"[SERVER] {msg}", file=sys.stderr, flush=True)

# The form the user will see. MCPServer turns this Pydantic model into the
# requestedSchema of the elicitation/create request (flat, primitive fields only).
class LaunchConfirmation(BaseModel):
    confirmLaunch: bool = Field(description="Confirm you want to launch this product")

```

```

price: float = Field(default=29.99, ge=0, description="Retail price (USD)")
initialStock: int = Field(default=100, ge=0, description="Initial stock quantity")
shippingSpeed: Literal["standard", "priority", "overnight"] = Field(default="standard",
description="Shipping speed to offer at launch")

LAUNCH_KEY = "launch_details" # key of the question in input_requests / input_responses

@mcp.tool()
async def launch_product(product_name: str, ctx: Context) -> str | InputRequiredResult:
    """Launch a product, confirming pricing and inventory with the user first."""
    # Protocol 2026-07-28 has no server->client back-channel, so a tool can't
    # `await ctx.elicit()`. Instead the body runs twice: round 1 returns the
    # question, the client answers and retries, round 2 reads the answer.
    answer = (ctx.input_responses or {}).get(LAUNCH_KEY)

    if answer is None:
        log(f"asking the user to confirm launch details for '{product_name}'")
        return InputRequiredResult(
            input_requests={
                LAUNCH_KEY: ElicitRequest(
                    params=ElicitRequestFormParams(
                        message=f"Please confirm launch details for '{product_name}':",
                        requested_schema=render_elicitation_schema(LaunchConfirmation),
                    )
                )
            }
        )

    # --- you are now "halfway" through the tool, with the user's answer in hand ---
    if answer.action == "decline":
        return "User declined to provide launch details."
    if answer.action != "accept":
        return "User cancelled the launch flow."

    data = LaunchConfirmation.model_validate(answer.content)
    if not data.confirmLaunch:
        return "Launch not confirmed by user."
    log("user accepted, finalizing launch")
    return (

```

```

    f"Launched {product_name}! Price: ${data.price:.2f}, "
    f"Initial stock: {data.initialStock}, Shipping: {data.shippingSpeed}."
)

if __name__ == "__main__":
    log("product-server starting on stdio...")
    mcp.run() # stdio transport by default

```

Aspect	Form mode	URL mode
Purpose	Collect structured data from the user	Send the user to a web page for a sensitive step
Direction	Server → Client → User (form inside the client)	Server → Client → User's browser (outside the client)
Who provides the answer?	The user, inside the client	The user, on the server's web page
Typical Use Case	Missing details, confirmations, choices	Third-party sign-in (OAuth), payments, entering API keys
What the server receives	The form data (on accept)	Only accept / decline / cancel — the data stays out of the client